TensorFlow RTX 50 Series — Conda + Docker Setup

Contents

1 Project Overview

This repository contains a complete configuration for running a **deep learning neural network** implemented in TensorFlow and trained on **NVIDIA GPUs** using **CUDA**. The application can run both on **GPU** and on **CPU**: at runtime, the logic in `main.py` automatically detects the available devices and decides whether to execute on a GPU (if present) or fall back to the CPU, printing a clear message in both cases.

The project is designed to run efficiently on modern GPUs, including the **RTX 50 Series (5060, 5070, 5080, 5090)**, which are not officially supported by the standard TensorFlow GPU wheels at the time of development (2025).

During the development of this application, TensorFlow did not provide precompiled kernels for `compute capability 12.0`. For this reason, the project relies on a **custom TensorFlow wheel** compiled specifically with RTX 50 support.

The repository supports **two ways** of running the project:

1. **Conda Environment (Local Development)** — runs directly on your host system and requires NVIDIA drivers, CUDA, and cuDNN installed on the host if you want GPU acceleration.
2. **Docker Container (Reproducible Environment)** — runs inside a container and can be executed either:
   - on an **NVIDIA GPU** (using the NVIDIA Container Toolkit, without manually installing CUDA/cuDNN on the host), or
   - on **CPU only**, without GPU acceleration and without any NVIDIA drivers.

In the Conda method, the script `install_tf.py` is used to detect the available GPU and install the appropriate TensorFlow build (custom RTX 50 wheel or standard upstream wheel). In the Docker method, TensorFlow is installed directly inside the image at build time via the Dockerfile, and `install_tf.py` is not used.

1.1 CPU/GPU Device Selection in `main.py`

The `main.py` file contains the logic that chooses the execution device at runtime. It uses TensorFlow to:

- list the available physical GPU devices,
- select `/GPU:0` if at least one GPU is present,
- otherwise fall back to `/CPU:0`,
- run a small matrix multiplication demo to verify that everything works correctly,
- print informative messages so the user knows whether the application is running on GPU or CPU.

This means that, once TensorFlow is installed correctly (via Conda or Docker), the application will automatically use the best available device without requiring manual configuration from the user.

---

2 Method 1 — Conda Environment (Local Development)

This method is intended for local development on your machine, using your host operating system. Here, GPU support depends on the **NVIDIA driver**, **CUDA**, and **cuDNN** installed on the host. If you only want to run on CPU, you can skip the CUDA/cuDNN installation and use a CPU-only TensorFlow build.

2.1 System Requirements: NVIDIA Drivers, CUDA and cuDNN (for GPU)

NVIDIA Drivers

To use the GPU on Linux or WSL2, you must have **NVIDIA Drivers 560+** installed (or newer, such as the 580 series).

```
nvidia-smi
```

System-Level CUDA and cuDNN

*Note: When running with the Conda method and using GPU acceleration, TensorFlow requires CUDA and cuDNN to be available at the **system level**, not only inside the virtual environment or Conda environment.*

Install CUDA 12.8 and cuDNN 9 (Ubuntu example):

```
sudo apt update
sudo apt install wget gnupg

# Add NVIDIA Repository
wget https://developer.download.nvidia.com/compute/cuda/repos/
   ubuntu2404/x86_64/cuda-ubuntu2404.pin
sudo mv cuda-ubuntu2404.pin /etc/apt/preferences.d/cuda-repository-pin
   -600

sudo apt-key adv --fetch-keys \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/
   x86_64/3bf863cc.pub

sudo add-apt-repository \
  "deb https://developer.download.nvidia.com/compute/cuda/repos/
   ubuntu2404/x86_64/ /"

# Install Toolkit and cuDNN
sudo apt update
sudo apt install -y cuda-toolkit-12-8
sudo apt install -y cudnn9-cuda-12
```

Update PATH and Libraries:

```
echo 'export PATH=/usr/local/cuda-12.8/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda-12.8/lib64:
   $LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc
```

2.2 Install Conda

If you do not have Miniconda installed:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64
    .sh -O miniconda.sh
bash miniconda.sh
source ~/.bashrc
```

2.3 Create the Environment

```
conda env create -f environment.yml
conda activate tf-project
```

2.4 TensorFlow Installation via `install_tf.py` (Local Only)

For `local Conda/venv environments`, this repository includes the script `install_tf.py`, which centralizes the TensorFlow installation logic. It performs the following steps:

- Detects the installed GPU model via `nvidia-smi`.
- If an **RTX 50 Series GPU** is detected → installs the custom TensorFlow wheel from mypapit/tensorflowRTX50.
- If a different (officially supported) NVIDIA GPU is detected → installs the official TensorFlow GPU build.
- If no compatible GPU is found → installs a CPU-only TensorFlow build.

Run the installer:

```
python install_tf.py
```

Note: `install_tf.py` is intended **only for local environments** (Conda or virtualenv). Docker images install TensorFlow directly via the `Dockerfile`.

2.5 Start the Project

After TensorFlow is correctly installed, you can run the main application:

```
python main.py
```

---

3 Method 2 — Docker (Recommended)

Recommended for grading, reproducible experiments, or users who do not want to manually install CUDA and cuDNN on the host system.

Docker allows you to isolate the environment and makes it easy to reproduce the same configuration on different machines.

In the Docker setup:

- TensorFlow is installed **at build time** inside the image.
- The script `install_tf.py` is **not used inside the container**.
- A build argument (`TF_VARIANT`) is used to choose between two image variants:
  - a **custom RTX 50 image** (using the custom wheel),
  - a **generic / official TensorFlow image** (for older GPUs or CPU-only).

3.1 Prerequisites

Docker Installation

You must have Docker (or Docker Desktop + WSL2 integration) installed on your system. Please refer to the official Docker documentation for your platform.

NVIDIA Container Toolkit (for GPU mode)

If you want to run the container on an NVIDIA GPU, install the **NVIDIA Container Toolkit** on the host:

```
sudo apt install nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

On Windows with Docker Desktop + WSL2, make sure:

- GPU support is enabled in Docker Desktop,
- WSL2 integration is enabled for your Ubuntu distribution.

Once configured, you can run containers with GPU access using `--gpus all`.

Dockerfile Overview

The provided `Dockerfile`:

- Uses `nvidia/cuda:12.8.0-cudnn-runtime-ubuntu24.04` as base image (CUDA + cuDNN already included).
- Creates a Python virtual environment at `/opt/venv`.
- Installs base project dependencies from `requirements.txt`.
- Uses a build argument `TF_VARIANT` to decide which TensorFlow variant to install:
  - `TF_VARIANT=rtx50` → installs the custom RTX 50 wheel and extra dependencies from `requirements-rtx50.txt`.
  - `TF_VARIANT=official` → installs the official TensorFlow release from PyPI.
- Sets `CMD ["python", "main.py"]` so the demo runs automatically when the container starts.

3.2 Build the Container Images

From the root of the repository (where the `Dockerfile` is located), you can build two images.

3.2.1 1. Custom RTX 50 Image (for RTX 5060 / 5070 / 5080 / 5090)

```
docker build -t tf-app-rtx50 --build-arg TF_VARIANT=rtx50 .
```

This image:

- installs the custom TensorFlow RTX 50 wheel,
- installs additional dependencies from `requirements-rtx50.txt`,
- is intended for RTX 50 series GPUs when run with `--gpus all`.

3.2.2 2. Official TensorFlow Image (for older GPUs or CPU)

```
docker build -t tf-app-official --build-arg TF_VARIANT=official .
```

This image:

- installs the official TensorFlow release from PyPI,
- can run on:
  - older RTX GPUs (when run with `--gpus all`),
  - CPU-only environments (when run without GPU flags).

3.3 Running the Container on GPU (NVIDIA)

To run the project using an NVIDIA GPU:

```
docker run --gpus all -it tf-app-rtx50
```

or, for the official TensorFlow image:

```
docker run --gpus all -it tf-app-official
```

In both cases:

- The container has access to your host GPU(s) via `--gpus all`.
- TensorFlow will attempt to use the GPU; if everything is configured correctly, `main.py` should detect at least one GPU and run on `/GPU:0`.

3.4 Running the Container on CPU Only

If you do not have a compatible NVIDIA GPU, or if you want to test CPU-only execution, you can start the official image without GPU flags:

```
docker run -it tf-app-official
```

Behavior:

- No GPU is exposed to the container.
- TensorFlow will run purely on CPU.
- `main.py` will print a message indicating that no GPU was found and it is running on `/CPU:0`.

---

4 Included Files

main.py	Main entry point for the deep learning application (model creation, training, evaluation, etc.). It also selects the execution device (GPU or CPU) at runtime and runs a small demo computation.
install_tf.py	Hardware-aware installer script. Detects GPU type and installs the appropriate TensorFlow wheel (custom RTX 50 or official build; GPU or CPU-only).
environment.yml	Conda environment definition used for local development (Method 1).

Dockerfile	Build instructions for the Docker image used in Method 2 (GPU and CPU modes).
requirements.txt	Python dependencies used in the Docker image and for pip-based installations.
README.md	High-level project documentation and usage instructions.

5 Final Notes

- At the time of development, the **RTX 50 Series** was not officially supported by standard TensorFlow GPU wheels, so a **custom build** is required.
- TensorFlow installation, adapting to:
  - RTX 50 Series GPUs (custom wheel),
  - Other NVIDIA GPUs (official GPU wheel),
  - CPU-only environments (no GPU detected).
- The **Conda method** is suitable for local development but requires NVIDIA drivers, CUDA, and cuDNN on the host if you want GPU acceleration.
- The **Docker method** is recommended for reproducibility and for users who want to avoid manual system-level CUDA/cuDNN setup. It can be run on:
  - GPU (via NVIDIA Container Toolkit and `--gpus all`), or
  - CPU only (no special host configuration required).
- For questions, suggestions, or issues, please open an Issue in the repository.